

Group Details

Abhinav Naidu (23CS02008)

Hari Nandan (23CS01048)

Sai (23CS01050)

Overview

GyanPustak is a web-based textbook marketplace tailored for college students, offering both purchase and rental options. The platform supports three categories of books—new, used, and electronic (eBooks), providing flexibility based on student needs and budgets.

The system features an intuitive, course-linked catalogue where books are organized according to university departments and courses, enabling students to easily discover required academic materials. Users can browse, search, add items to their cart, and complete transactions seamlessly.

GyanPustak operates with two primary user groups: students and employees. Employees are further categorized into customer support users, administrators, and a super-administrator, each with distinct roles and permissions.

In addition to managing the full lifecycle of book transactions, from browsing and ordering to fulfillment and shipping. The platform incorporates a structured ticketing system. This allows users to raise issues and enables employees to efficiently track, manage, and resolve them.

Assumptions

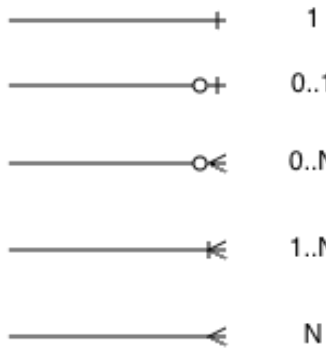
1. Platform accessible via browsers like Google Chrome and Internet Explorer
2. No book return policy; only pre-fulfillment cancellations allowed
3. One active cart per student at any time
4. All new tickets start with status 'new'
5. Employee Aadhaar numbers stored securely as ID reference
6. Exactly one super-administrator with highest privileges
7. Books linked to courses using university, department, semester, and year
8. Book ratings stored as aggregated values from student reviews

GitHub Repository

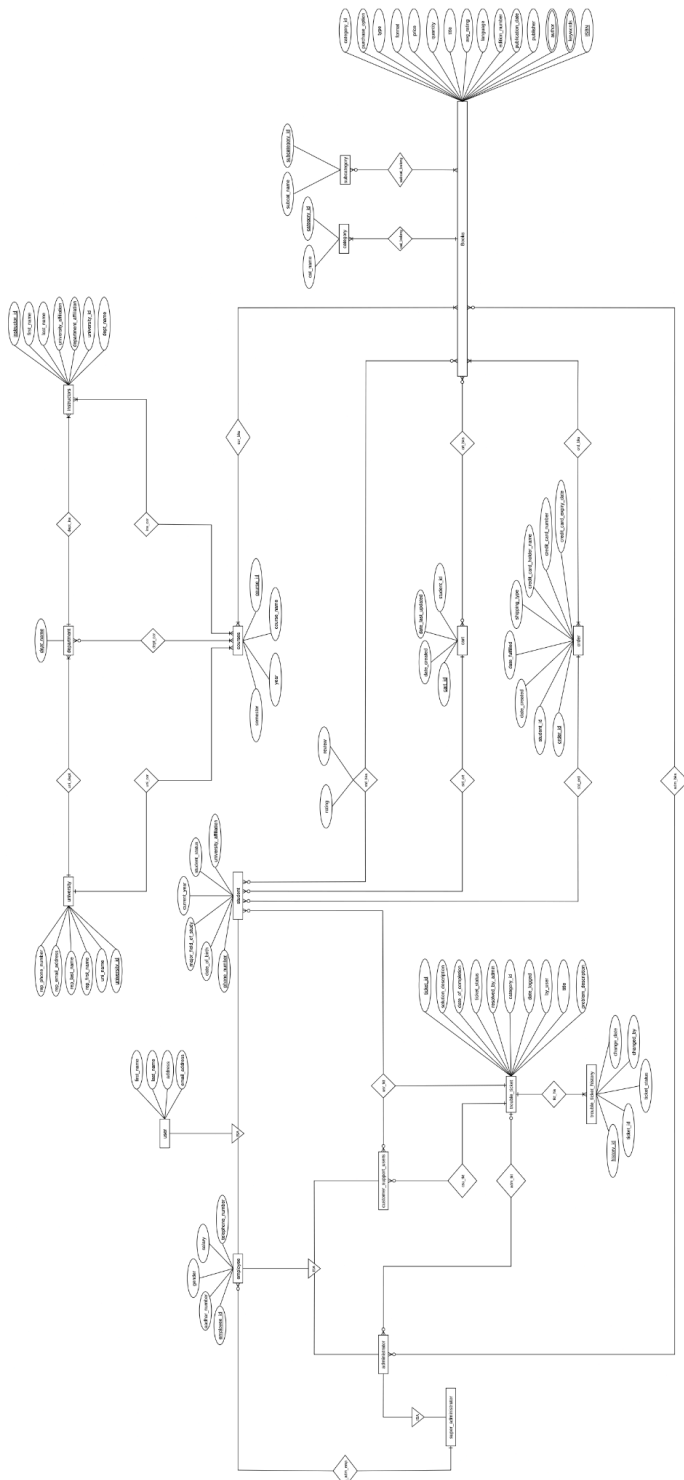
The Python Implementations were in the [GitHub](#) link.

Notations

- a. Crow's foot notation - Used for ER Diagrams



Full ER Diagram - [ER_Diagram.pdf](#)



ER Diagram Relations

1. University - Department

university (1..1) - <uni_dept> - department (1..N)

2. University - Courses

university (1..1) - <uni_cor> - courses (1..N)

3. Department - Instructors

department (1..1) - <dept_ins> - instructors (1..N)

4. Instructors - Courses

instructors (1..N) - <ins_cor> - courses (1..N)

5. Department - Courses

department (0..N) - <dept_cor> courses (1..N)

6. Courses - Books

courses (1..N) - <cor_bks> - books (1..N)

7. Student - Books

student (0..N) - <std_bks> - books (0..N)

8. Student - Cart

student (0..1) - <std_crt> - cart (1..1)

9. Student - Order

student (0..N) - <std_ord> - order (1..1)

10. Cart - Books

cart (0..N) - <crt_bks> - books (0..N)

11. Order - Books

order (1..N) - <ord_bks> - books (1..N)

12. Student - Trouble_Ticket

student (0..N) - <usr_tkt> - trouble_ticket (1..1)

13. Customer_Support_Users - Trouble_Ticket

customer_support_users (0..N) - <usr_tkt> - trouble_ticket (1..1)

customer_support_users (0..N) - <csu_tkt> - trouble_ticket (1..1)

14. Administrator - Trouble_Ticket

administrator (0..N) - <adm_tkt> - trouble_ticket (0..1)

15. Administrator - Books

administrator (0..N) - <adm_bks> - books (0..N)

16. Super_Administrator - Employee

super_administrator (1..1) - <s_adm_emp> - employee (0..N)

17. Trouble_Ticket - Trouble_Ticket_History

trouble_ticket (1..1) - <tkt_his> - trouble_ticket_history
(1..N)

18. Books - Category

book (1..1) - <cat_belong> - category (1..N)

19. Books - Subcategory

book (1..N) - <subcat_belong> - subcategory (0..N)

20. Books - Author

book (1..N) - <auth_rel> - authors (0..N)

21. Books - Keyword

book (1..N) - <keyw_rel> - keywords (0..N)

Relational Schema

1. *user* (user_id (pk), password, first_name, last_name, phone_number, address, email_address)
 - a. *employee* (employee_id (pk), user_id (fk), aadhar_number, gender, salary)
 - i. *administrator* (employee_id (pk), department, employee_id (fk))
 1. *super_administrator* (admin_id (pk), employee_id (fk))
 - ii. *customer_support_users* (employee_id (pk), employee_id (fk), specialisation_area)
2. *student* (student_id (pk), user_id (fk), phone_number, date_of_birth, university_name, major_field_of_study, current_year, student_status)
3. *books* (ISBN (pk), category_id (fk), purchase_option, type, format, price, rent_price, quantity, title, avg_rating, language, edition_number, publication_date, publisher, description)
4. *university* (university_id (pk), uni_name, rep_first_name, rep_last_name, rep_email_address, rep_phone_number)
5. *department* (university_id (fk), dept_name (pk), dept_id (pk))
6. *instructors* (instructor_id (pk), university_id (fk), dept_name, first_name, last_name, university_id, department_id)
7. *courses* (course_id (pk), university_id (fk), department_id, course_code, course_name, year, semester)
8. *trouble_ticket* (ticket_id (pk), solution_description, date_of_completion, ticket_status, resolved_by_admin_id (fk), category, date_logged, created_by_user_id (fk), title, problem_description)
9. *cart* (cart_id (pk), student_id (fk), date_created, date_last_updated)
10. *order* (order_id (pk), student_id (fk), date_created, date_fulfilled, shipping_type, credit_card_holder_name, credit_card_number, credit_card_expiry_date, credit_card_type, order_status, amount)
11. *trouble_ticket_history* (history_id (pk), ticket_id (fk), changed_by_user_id (fk), change_date, old_status, new_status, changed_at)
12. *category* (category_id (pk), parent_id (fk), cat_name)
13. *author* (author_id (pk), book_id (fk), PRIMARY KEY (author_id, book_id))
14. *keywords* (keyword_id (pk), book_id (fk), keyword, PRIMARY KEY (keyword_id, book_id))

15. *dept_cor* (dept_id (fk), course_id (fk), PRIMARY KEY (dept_id, course_id))
16. *ins_cor* (instructor_id (fk), course_id (fk), PRIMARY KEY (instructor_id, course_id))
17. *cor_bks* (course_id (fk), ISBN (fk), PRIMARY KEY (course_id, ISBN))
18. *std_bks* (review_id (pk), student_id (fk), ISBN (fk), rating, review)
19. *crt_bks* (cart_id (fk), ISBN (fk), quantity, purchase_option, PRIMARY KEY (cart_id, ISBN))
20. *ord_bks* (order_id (fk), ISBN (fk), quantity, price, purchase_option, PRIMARY KEY (order_id, ISBN))

Normalization

First Normal Form (1NF) — Atomicity

- All attributes are **atomic**, with no repeating groups or multi-valued fields.
- Multi-valued attributes such as *Authors*, *Keywords*, and *Categories* are decomposed into separate associative (bridge) tables:
 - `auth_bks`
 - `keyw_bks`
- Each table contains only indivisible values, ensuring compliance with 1NF.

Second Normal Form (2NF) — Full Functional Dependency

- All non-key attributes are **fully functionally dependent** on the entire primary key.
- This is particularly relevant for tables with composite keys.
- Example:
 - In `cart(cart_id, book_id)`, attributes such as `quantity` and `rent_duration` depend on the **entire composite key**, not just a subset.
- Partial dependencies are eliminated across the schema.

Third Normal Form (3NF) — Elimination of Transitive Dependency

- No non-key attribute depends on another non-key attribute.
- Each non-key attribute depends **only on the primary key**.
- Example:
 - Publisher-related information is stored directly within the **Book** table, avoiding indirect dependencies through another non-key attribute.
- This ensures reduced redundancy and prevents update anomalies.

Boyce–Codd Normal Form (BCNF) — Stronger Dependency Constraints

- Most relations satisfy BCNF, where **every determinant is a candidate key**.
- Example:
 - In the `Instructor` entity, attributes such as `university_id` and `dept_id` are directly associated with the primary key and are not derived from other non-key attributes.
- The **ISA hierarchy** (Employee → Admin / CSU) further enforces normalization by:
 - Separating role-specific attributes
 - Eliminating redundancy
 - Maintaining clear entity boundaries

Workflow

User Role Permissions - The system defines four primary roles: *Student*, *Customer Support*, *Admin*, and *Super Admin*. Each role has clearly defined capabilities to ensure secure and structured access control.

Capability	Student	Customer Support	Admin	Super Admin
Browse / Search Books	✓	✓	✓	✓
Place Orders	✓	x	✓	✓
Write Reviews	✓	x	✓	✓
Create Tickets	✓	✓	x	✓
Edit 'New' Tickets	x	✓	x	✓
Handle Assigned Tickets	x	x	✓	✓
Update Book Inventory	x	x	✓	✓
Add Course-Required Books	x	x	✓	✓
Add New Employees	x	x	x	✓

- *super_admin* has full system control.
- *admin* handles operational tasks (book-inventory, tickets).
- *customer_support_users* acts as an intermediary for ticket creation and escalation.
- *students* interact mainly with books, orders, and tickets.

Order Lifecycle - The order processing system in GyanPustak follows a linear and state-driven workflow, ensuring clarity and consistency in transaction handling.

Cart → Checkout → Order Created ('new') → Processed → Awaiting Shipping → Shipped

- Each student maintains **one active cart** (`cart_id` uniquely mapped to a student).
- During checkout:
 - Cart contents are transferred to the **Order** and **Order_Items** tables.
 - The cart is cleared for future use.

Order Status Progression

new → processed → awaiting shipping → shipped

Payment & Shipping

Credit card details stored - Card number, Expiry date, Cardholder name, Card type

Shipping options - standard, 2-day, 1-day

Cancellation Policy

- Orders can be canceled **only before the 'shipped' stage**.
- On cancellation, order status is updated to `canceled`
- No return mechanism is supported.
- The system supports **cancellation only**, not post-delivery returns.

Ticket Lifecycle - The trouble ticket system is designed with a controlled state transition model and strict role-based access.

Ticket Workflow

new → assigned → in-process → completed

State Descriptions

new	1. Created by <i>student</i> or <i>customer_support_user</i> 2. Visible and editable only by <i>customer_support_user</i>
assigned	1. Ticket is escalated by <i>customer_support_user</i> 2. Assigned to an <i>admin</i> 3. <i>admin</i> gains access
in_process	1. <i>admin</i> begins working on the issue 2. Status updated to reflect active handling
completed	1. <i>admin</i> resolves the ticket

Implementation

Few reference for Tables -

1. Database Creation

```
CREATE DATABASE IF NOT EXISTS gyanpustak;  
USE gyanpustak;
```

2. User

```
CREATE TABLE IF NOT EXISTS users (  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    role ENUM('student', 'support', 'admin', 'superadmin') NOT NULL,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    phone VARCHAR(20),  
    address TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3. Students

```
CREATE TABLE IF NOT EXISTS students (  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL UNIQUE,  
    date_of_birth DATE,  
    university VARCHAR(255),  
    major VARCHAR(255),  
    student_status ENUM('graduate', 'undergraduate') DEFAULT 'undergraduate',  
    year_of_study INT,  
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE  
);
```

4. Employees

```
CREATE TABLE IF NOT EXISTS employees (  
    employee_id VARCHAR(50) PRIMARY KEY,  
    user_id INT NOT NULL UNIQUE,  
    gender ENUM('male', 'female', 'other'),  
    salary DECIMAL(12, 2),  
    aadhaar VARCHAR(20),  
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE  
);
```

5. Customer Support Staff

```
CREATE TABLE IF NOT EXISTS support_staff (  
    employee_id VARCHAR(50) PRIMARY KEY,  
    shift ENUM('morning', 'evening', 'night') DEFAULT 'morning',  
    specialization_area VARCHAR(100) DEFAULT 'general',  
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id) ON DELETE CASCADE  
);
```

6. Administrator Staff

```
CREATE TABLE IF NOT EXISTS admin_staff (  
    employee_id VARCHAR(50) PRIMARY KEY,  
    department VARCHAR(100) DEFAULT 'operations',  
    admin_level INT DEFAULT 1,  
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id) ON DELETE CASCADE  
);
```

Few reference for Queries -

1. Top 5 Highest-Rated Books with Review Count

```
mysql> SELECT  
->     b.book_id,  
->     b.title,  
->     b.avg_rating,  
->     COUNT(r.review_id) AS review_count  
-> FROM books b  
-> LEFT JOIN reviews r  
->     ON b.book_id = r.book_id  
-> GROUP BY b.book_id, b.title, b.avg_rating  
-> HAVING COUNT(r.review_id) > 0  
-> ORDER BY b.avg_rating DESC, review_count DESC  
-> LIMIT 5;
```

2. All Unresolved Tickets Escalated to Admin (Status = 'assigned' or 'inprocess')

```
mysql> SELECT  
->     tt.ticket_id,  
->     tt.title,  
->     tt.problem_description,  
->     tt.category,  
->     tt.status,  
->     tt.date_logged,  
->     tt.assigned_admin_id,  
->     CONCAT(u.first_name, ' ', u.last_name) AS admin_name,  
->     u.email AS admin_email  
-> FROM trouble_tickets tt  
-> LEFT JOIN users u  
->     ON tt.assigned_admin_id = u.user_id  
-> WHERE tt.assigned_admin_id IS NOT NULL  
->     AND tt.status IN ('assigned', 'in-process')  
-> ORDER BY tt.date_logged DESC;
```

Conclusion

GyanPustak successfully models a comprehensive and structured textbook marketplace tailored to the needs of college students. By integrating book purchasing and rental services with an academic course-based catalogue, the platform ensures that students can easily access relevant study materials. The system design effectively captures multiple real-world entities such as students, employees, books, universities, and courses, while maintaining clear relationships and constraints between them. Additionally, features like a single active cart, structured order lifecycle, and aggregated book ratings contribute to a streamlined and user-friendly experience.

The platform also emphasizes operational efficiency through a well-defined role hierarchy consisting of customer support users, administrators, and a super-administrator. This separation of responsibilities ensures proper handling of tasks such as ticket management, inventory updates, and employee administration. The inclusion of a robust trouble ticketing system further enhances the platform by enabling systematic issue tracking and resolution, thereby improving reliability and user satisfaction.

Overall, GyanPustak demonstrates a well-balanced integration of database design, application logic, and real-world usability. The system not only fulfills the functional requirements of an academic marketplace but also provides a scalable foundation for future enhancements such as advanced analytics, recommendation systems, or expanded institutional integrations.